

ReGNN: A Redundancy-Eliminated Graph Neural Networks Accelerator

Cen Chen^{†‡}, Kenli Li^{*†}, Yangfan Li[†], Xiaofeng Zou[†]^{*}Corresponding Author[†]College of Information Science and Engineering, Hunan University, China[‡]Shenzhen Institute, Hunan University, China

{chencen, lkl, yangfanli, zouxiaofeng}@hnu.edu.cn

Abstract—Graph neural networks (GNNs), which extend conventional deep learning technologies to process graph-structured data, have shown its powerful graph representation learning ability. Existing typical GNNs utilize neighborhood message passing mechanism based on neural networks that updates target vertex representations by aggregating feature messages from neighboring source vertices. To accelerate the computations of GNNs, some customized accelerators, which follow the neighborhood aggregation computation pattern for each vertex, have been proposed. Through analysis, we observe that a naive implementation of the neighborhood aggregation results in redundant computations and communications.

In this paper, we propose a novel redundancy-eliminated GNN accelerator, shortly termed as ReGNN. ReGNN is supported by an algorithm and architecture co-design. We first propose a dynamic redundancy-eliminated neighborhood message passing algorithm for GNNs. Then a novel architecture is designed to support the proposed algorithm and transform the redundancy elimination into performance improvement. ReGNN is also a configurable pipelined architecture that can be configured to support different GNN variants. In terms of the same computations, ReGNN provides the same accuracy as traditional GNNs. To the best of our knowledge, ReGNN is the first accelerator that can eliminate computation redundancy in GNNs. Our proposed ReGNN system gains an average of $9.1\times$ speedup and $8.9\times$ energy efficiency over state-of-the-art GNN accelerators.

Keywords—Graph Neural Networks, Hardware Accelerator, Redundancy-Aware Computation, Software-Hardware Co-Design

I. INTRODUCTION

Recently, graph neural networks (GNN), based on deep learning technologies for graphing structured data, have achieved superior performance across various graph-based tasks, such as computer vision, natural language processing and recommendation systems [29], [37]. Existing typical GNNs are primarily based on a recursive neighborhood message passing scheme, where each vertex receives features of its neighbors and uses the received messages to update its own features within a single layer of a GNN [3], [8], [29]. Multiple neighborhood aggregation is implemented recursively so that vertices can collect information from other vertices that are in its k -hop network neighborhood. The final features are subsequently used for downstream prediction tasks, including graph classification [36], node classification [17] and link prediction [35].

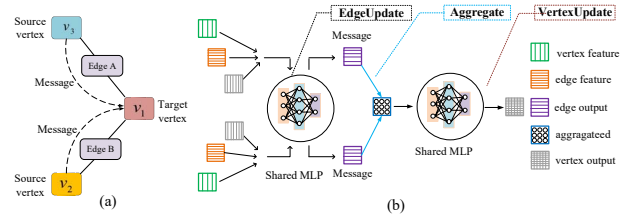


Fig. 1. (a) Neighborhood message passing mechanism of GNNs. (b) Computing abstraction of neighborhood message passing mechanism.

Scaling GNNs to efficiently process large real-world graph datasets remains a big challenge [22]. For example, the e-commerce graph data contains billions of users and items. In particular, many implementations aim to work with the entire graph Laplacian matrix, which cannot fit into main memory even for medium-size graphs [17]. To solve this problem, some studies abstract the dataflow of a message passing layer using the vertex-program abstraction for GNNs [6], [21], [22] into three important stages: EdgeUpdate, Aggregation and VertexUpdate, which is shown in Fig. 1. In general, EdgeUpdate computes the messages for each edge through utilizing neural networks and Aggregate gathers neighboring messages for each target vertex. Ultimately, VertexUpdate combines the old vertex features of each target vertex and the aggregated features to produce new vertex features of each target vertex using neural networks. Different variants of GNNs have different combinations of these stages (the details of the abstraction are presented in Section II-A). The EdgeUpdate and VertexUpdate phase act as neural networks, which are usually represented by a matrix-vector multiplication (MVM). The Aggregation phase maintains most graph processing behaviors and it heavily relies on the graph structure that is inherently random and sparse. Processing of each target vertex requires aggregating features from all its source neighbors.

The hybrid computing pattern discussed above which involves both regular computations, and a large amount of irregular execution patterns, which exerts negative influence on the CPU and GPU [21], [31]. Specialized hardware architecture is a promising option to improve the computational efficiency of GNNs. However, previous graph processors and neural network accelerators can only be optimized to support either graph processing or neural networks, rather than both of them simultaneously. To address this, several customized

accelerators [1], [7], [16], [20], [21], [31] are proposed, following the computational abstraction of neighborhood message passing mechanism. They have highlighted the importance of designing new customized accelerators for GNNs, which achieved a significantly higher performance with reduced energy consumption than GPUs and existing DNN accelerators for GNNs.

The aforementioned accelerators utilize vertex-program abstraction and conduct neighborhood aggregation for each vertex independently. In this work, we argue that the way of conducting neighborhood message passing for each vertex independently introduces heavy redundant computations for EdgeUpdate and Aggregation. This is because when vertices share some same neighbors, the intermediate results of these neighbors generated in the EdgeUpdate for a specific vertex can be reused for other vertices with the same neighbors. If some vertices have the same set of neighbors, the intermediate aggregated results of the same neighbors of a specific vertex can also be reused during the Aggregation phase of other vertices. However, existing GNNs accelerators tend to ignore this characteristic of GNNs, thus involving significant redundant computations during executions. The number of feature dimensions of each vertex is generally large. e.g., 512, 1024, 2048. As a result, EdgeUpdate computations (MVM) are expensive. Moreover, the overhead of transferring features of vertices from off-chip memory to on-chip memory is large. Hence, how to improve the reuse of data in the on-chip memory is critical for improving the performance.

To eliminate the redundant computations in GNNs, we first propose a dynamic redundancy-eliminated message passing algorithm for GNNs to eliminate computational redundancies of both EdgeUpdate and Aggregation in one phase. The proposed algorithm can identify and eliminate computation redundancies, performs the same computations and offers the same models/accuracy as traditional GNNs. Although the redundant computations can be eliminated by the proposed algorithm, it incurs significant irregular computing and memory patterns, which significantly hinders an efficient architecture design. To tackle this, we propose a customized efficient redundancy-eliminated GNN accelerator, termed as ReGNN, with which the redundancy elimination can be turned into performance improvement. ReGNN is designed to support the proposed dynamic redundancy-eliminated message passing algorithm efficiently and further alleviate these irregularities. ReGNN can also help schedule the irregular redundancy-eliminated tasks on-the-fly to maximize parallelism via the proposed dynamic dispatching algorithm. Moreover, through several design mechanisms, such as redundancy-aware bulk execution, communication-efficient prefetch, neighbor shuffling schemes, and configurable cache management schemes, ReGNN can maximize the data reuse of original vertices features, intermediate results of EdgeUpdate and Aggregate to reduce the Communication redundancies.

Another goal in this work is to enable the architecture to support representative variants of GNNs. To this end, we design a configurable pipelined architecture that can assist in

eliminating computation redundancy of typical GNNs algorithms by control command.

To summarize, we have made the following contributions:

- We provide an analysis of the redundant computations in GNNs and propose a dynamic redundancy-eliminated message passing algorithm for GNNs.
- We further propose a novel architecture design to help identify and eliminate redundant computations and further reduce the redundant communications. Moreover, we propose a redundancy-aware dynamic dispatch mechanism, which elaborately schedules redundancy-eliminated tasks on the fly, effectively tackling computation irregularities and exploiting the intra-task regularity with highly parallelized computing engines.
- Unlike some existing state-of-the-art accelerators, including HyGCN [31], AWB-GCN [7] and GCNAX [20], which only support the acceleration of GCNs, ReGNN is a flexible architecture that can support various GNN algorithms with user-defined configuration.
- Compared with HaGNN, the state-of-the-art accelerator for GNNs, ReGNN gains an average of $10.3\times$ speedup and $8.7\times$ energy-efficiency. Compared with HyGCN, the accelerator for GCNs, ReGNN gains an average of $9.7\times$ speedup and $9.8\times$ energy-efficiency for GCNs.

II. BACKGROUND AND ANALYSIS

A. Graph Neural Networks and Abstraction

Unlike convolutional neural networks (CNNs) that mainly deal with Euclidean data like images and videos, GNNs generalize the neural networks to operate on graph structure data [17]. Recently, various GNN architectures have been proposed, like [3], [10], [17], [29], [30], [37]. Although variants of GNNs have been proposed, message passing algorithms [8] act as the key operation of all GNNs that update vertex representations by aggregating feature messages from neighboring vertices. Based on the common pattern observed in GNN models, [6], [22], [29] abstract the computation model in a message passing layer into three important stages: EdgeUpdate, Aggregate and VertexUpdate. The processing stages are listed as follows:

- EdgeUpdate. It computes the message for each edge by utilizing neural networks.
- Aggregate. This operation gathers the neighbor information for each vertex and reduces it to a single feature vector.
- VertexUpdate. Utilizing neural networks, it creates a new vertex feature by combining the old vertex features and the aggregated features.

An illustrative example of GNNs is shown in Fig. 1. After k iterations of EdgeUpdate, Aggregate and VertexUpdate, a vertex is represented by its final feature vector, which captures the structure and feature information within the vertex's k -hop neighborhood. For different variants of GNNs, their specific computations for EdgeUpdate, Aggregate and VertexUpdate also vary.

TABLE I

GNN ALGORITHMS ON NEIGHBORHOOD MESSAGE PASSING MODEL AND REDUNDANCY ANALYSIS. σ DENOTES THE ACTIVATION FUNCTION, W DENOTES THE MODEL WEIGHTS, $\mathcal{N}(v)$ DENOTES THE NEIGHBOR SET OF VERTEX v , $e_{u,v}$ MEANS THE WEIGHT OF EDGE (u, v) , a_v^k REPRESENTS THE AGGREGATION RESULTS FOR VERTEX v , h_v^k DENOTES THE FEATURE VECTOR OF VERTEX v AT THE k -TH LAYER, EDGEUPDATE REDUNDANCY (ER) AND AGGREGATION REDUNDANCY (AR).

Algorithms	EdgeUpdate	Aggregate	VertexUpdate	ER	AR
GCN [18]	Null	$a_v^k = \sum_{u \in \mathcal{N}(v)} h_u^{k-1}$	$h_v^k = \sigma(W^k \cdot a_v^k)$	No	Yes
GIN [30]	Null	$a_v^k = \sum_{u \in \mathcal{N}(v)} h_u^{k-1}$	$h_v^k = \sigma(W^k \cdot a_v^k)$	No	Yes
SGC [28]	Null	$a_v^k = \sum_{(u,v) \in \mathcal{E}^k} h_u^0$	$h_v^k = \sigma(W^k \cdot a_v^k)$	No	Yes
GraphSage-Pool [10]	$e_{u,v} = \sigma(W_1^k \cdot h_u^{k-1})$	$a_v^k = \max_{u \in \mathcal{N}(v)} \{e_{(u,v)}\}$	$h_v^k = \sigma(W_2^k \cdot a_v^k)$	Yes	Yes
G-GCN [4]	$e_{u,v} = \sigma(W_1^k \cdot h_u^{k-1} + W_2^k \cdot h_v^{k-1})$	$a_v^k = \sum_{u \in \mathcal{N}(v)} e_{u,v} \odot h_u^{k-1}$	$h_v^k = \sigma(W_3^k \cdot a_v^k)$	Yes	No

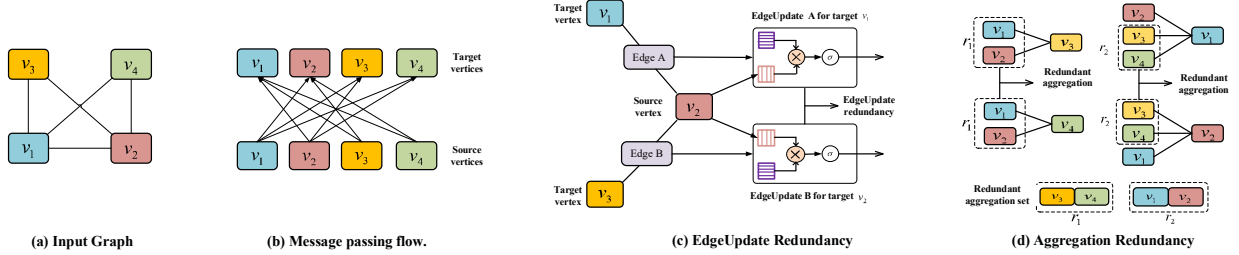


Fig. 2. Analysis of message passing mechanism based on an illustrative example. (a) An illustrative input graph. (b) Execution flow of the message passing process. (c) Illustrative examples of EdgeUpdate redundancy (ER). (d) Illustrative examples of Aggregation redundancy (AR).

GNNs generally fall into two categories based on the combination of the aforementioned execution phases [22]: those without EdgeUpdate and those with EdgeUpdate. As for the first type, a typical algorithm is GCN [18], which only involves an Aggregation phase for neighbors of each vertex and a Update phase for each vertex based on the aggregated features. Regarding the second type, a typical algorithm is GraphSage [27], which involves an additional phase prior to Aggregation: neural networks are utilized to update the messages for each edge. Table I lists the computations for several popular GNN algorithms in detail.

B. Analysis of Redundancy in GNN

One of the most important characteristics of graph data is that the structure of graph is irregular and the vertices may have abundant overlapped neighbors. Because of this, a naive implementation of the message passing model can lead to redundant EdgeUpdate and Aggregate computations, and communications.

1) Communication Redundancy

Given the input graph shown in Fig. 2(a), the message passing flow of a GNN layer for this graph is shown in Fig. 2(b). For example, messages from source vertices (v_2, v_3, v_4) should be passed to target vertex v_1 , whilst those from source vertices (v_1, v_3, v_4) should be passed to target vertex v_2 . During the execution for target vertex v_1 , features of (v_1, v_2, v_3, v_4) are required to be sent to the on-chip memory for execution, while for target vertex v_2 , features of (v_1, v_2, v_3, v_4) need to be sent to the on-chip memory. If v_1 and v_2 are executed separately, features of (v_1, v_2, v_3, v_4) should be sent to the on-chip memory for execution twice.

2) EdgeUpdate Redundancy

Through analyzing the computing model of a message passing layer, we find that if some target vertices have overlapping

source vertices, there will be redundant EdgeUpdate operations when performing EdgeUpdate for the edges between target vertices and overlapping source vertices. As shown in Fig. 2(c), target vertices v_1 and v_3 have the same neighbor v_2 . A separate implementation of aggregating features for v_1 and v_3 involves conducting EdgeUpdate operations for v_2 twice. Apparently, such process creates one redundant EdgeUpdate operation for v_2 .

3) Aggregation Redundancy

We also observe that there is considerable redundancy when aggregating the overlapping neighbors. An example is shown in Fig. 2, where some vertices have some common neighbors. Specifically, v_1 and v_2 share a set of neighbors (v_3, v_4) , while v_3 and v_4 similarly have a shared set of neighbors (v_1, v_2) . Therefore, if we conduct Aggregation operations for each vertex separately, both (v_3, v_4) and (v_1, v_2) are aggregated twice. This evidently makes Aggregation operations for (v_3, v_4) and (v_1, v_2) redundant.

Real-world graphs often possess clustering characteristics and has many dense subgraphs [12], so we suggest that the Aggregation redundancy can cause heavy computations in GNNs. Furthermore, as GNNs become wider and multi-layered, the redundancies in GNNs account for a significant fraction of all computations. Specifically, our experiments show that in modern GNNs, up to 84% of the aggregations are redundant and can be avoided to speed up training as well as inference of GNNs.

C. Opportunities

The number of feature dimensions of each vertex is generally large, e.g., 512, 1024, 2048. EdgeUpdate computations (matrix-vector multiplication) are costly. Generally, the number of edges is much larger than that of vertices. Therefore, reusing intermediate results to avoid EdgeUpdate redundancy

brings many benefits. Meanwhile, the overhead of transferring features of vertices from off-chip memory to on-chip memory is also large. Therefore, reusing transferred features of vertices and intermediate results to mitigate the Communication redundancy can improve the performance.

Note that, an aggregation set contains many original vertices. The original approach is to transfer feature vectors of all these vertices to on-chip memory, and then conduct EdgeUpdate and Aggregation for these vertices. Actually, the intermediate aggregation result of an aggregation set is only a feature vector, rather than multiple feature vectors. Therefore, reusing intermediate results of aggregation sets and avoiding aggregation redundancy can reduce not only aggregation computations but also off-chip communications.

D. Overview of Our Solutions

ReGNN is supported by an algorithm and architecture co-design. (i) We first design a dynamic redundancy-aware graph representation so that the algorithm can identify the redundancies of EdgeUpdate and Aggregation. Based on the designed graph representation, an optimized algorithm is proposed to eliminate the EdgeUpdate and Aggregation redundancies for the message passing process in GNNs, and to minimize the Communication redundancy. Moreover, our proposed algorithm can alleviate the EdgeUpdate and Aggregation computing irregularities which improves the inter- and intra-process parallelism. (ii) ReGNN is an accelerator architecture for low-latency GNN execution that can support the proposed dynamic redundancy-eliminated message passing algorithm. Moreover, ReGNN can be adapted to support different variants of GNNs.

III. REDUNDANCY-ELIMINATED NEIGHBORHOOD MESSAGE PASSING

This section presents our proposed dynamic redundancy-eliminated message passing algorithm based on our design of redundancy-aware graph structure, which can eliminate redundancies during one message passing process.

A. Redundancy-aware Graph Structure Design

1) Redundant Aggregation Set

First, redundant aggregation set is defined to represent a common set of neighbors. Given a graph (V, E) , V is the vertex set in the graph, E is the edge set, the definition of a redundant aggregation set is formally presented as follows.

Definition 1 (Redundant Aggregation Set): Given a vertex set R , $R_i \in V$, if it exists an another set of target vertices $\hat{V}$ that have common set of neighbors: R , i.e., $R, \hat{V} \in V$ and $|R|, |\hat{V}| > 1$, satisfying $\forall u \in R, \forall v \in \hat{V}$ and the edge $e = (u, v) \in E$, then R can be defined as a redundant aggregation set.

We take the graph in Fig. 2 as an example. Given a vertex set $r = \{v_3, v_4\}$, and the target vertex set $\hat{r} = \{v_1, v_2\}$ exists, according to the definition, $\{v_3, v_4\}$ is a redundant aggregation set. Given the graph G , we adopt the search algorithm in [13] to search redundant aggregation sets in G .

If the intermediate results of these redundant aggregation sets can be stored after the first aggregation, the aggregation

results can be reused in the subsequent message passing processes.

2) Design of Redundancy-aware Graph Structure

In a graph, one target vertex may have some neighboring vertices. We treat all the redundant aggregation sets as independent neighbors to replace all the vertices in the corresponding redundant aggregation set. Therefore, the neighbors of vertices in V can be grouped into two categories: redundant aggregation sets and original vertices. One redundant aggregation set contains a set of original vertices.

To distinguish between redundant aggregation sets and original vertices, each redundant aggregation set or each original vertex is associated with an ID number and an ID type. The ID type is represented as a bit to denote that it is a redundant aggregation set or an original vertex.

To identify the EdgeUpdate and Aggregation redundancy, each original vertex is associated with a status bit, which denotes whether the original vertex has been conducted EdgeUpdate operations or not. Each redundant aggregation set is also associated with a status bit, which denotes whether the redundant aggregation set has been conducted Aggregate operations or not.

B. Dynamic Redundancy-eliminated Message Passing

1) Necessities

Based on the redundancy-aware graph structure, we can identify the redundant aggregation sets, and the intermediate results of EdgeUpdate and Aggregate. A straightforward way of eliminating redundancies on prior accelerators is to modify the accelerators so that they can store the intermediate results of EdgeUpdate and Aggregate, and then adopt multiple separate phases using software control to memorize intermediate results and further reuse them to avoid EdgeUpdate and Aggregate redundancies, which are presented as follows. (i) First, transfer all the original vertices from the off-chip memory to the on-chip memory, conduct EdgeUpdate operations for these vertices and then store the intermediate results in the off-chip memory. (ii) Transfer all the redundant sets from the off-chip memory to the on-chip memory, conduct Aggregate operations and store the intermediate results in the off-chip memory. (iii) Conduct message passing for each target vertex by reusing the intermediate results of EdgeUpdate and Aggregate.

This straightforward solution has the following drawbacks. (i) Due to the complex structure of graphs, the three processes cannot be pipelined. In other words, the subsequent phase has to wait until the completion of the previous process. This greatly affects the execution efficiency. (ii) During each phase, we need to transfer the vertex features from the off-chip memory to the on-chip memory for execution, which introduces large-scale redundant communications. (iii) The results required to be stored are intermediate results of EdgeUpdate or Aggregate. The whole message passing execution pipeline includes EdgeUpdate, Aggregate and VertexUpdate. Nevertheless, the first phase only involves EdgeUpdate, while the second phase only involves Aggregate. This characteristic makes some computing resources idle during the execution of

the first two aforementioned phases. For example, during the first phase, the Aggregate computing units are idle.

2) Challenges

To address these issues, we propose a dynamic redundancy-eliminated message passing algorithm, which aims at eliminating redundancies during one message passing process. However, many challenges and issues need to be considered when designing the algorithm.

First, identifying, storing and reusing intermediate results to eliminate redundancies during an individual message passing process incurs significant irregular computing and memory patterns, both of which increase the difficulty for designing efficient architecture. The neighbors of a target vertex include both redundant aggregation sets and original vertices, which introduces irregular computation patterns of EdgeUpdate and Aggregation. (i) EdgeUpdate irregularities. For a specific target vertex, EdgeUpdate vertices are only needed for original vertices, rather than redundant aggregation sets. Moreover, neighbors that have intermediate results and those without intermediate results require different computation patterns. For the first type, we only need to fetch the stored results, while for the second type, additional computations are required. (ii) Aggregation irregularities. After the EdgeUpdate execution, we need to aggregate all the neighbors, including both redundant aggregation sets and original vertices, for a specific target vertex. As the intermediate results of redundant aggregation sets can be reused in the subsequent message passing processes, we need to conduct aggregation for redundant aggregation sets and store the intermediate results.

Second, as previously discussed, the number of feature dimensions of each vertex is generally large, e.g., 512, 1024, 2048. Communications between off-chip and on-chip memory introduces large overhead. Therefore, we should consider reusing the features of vertices, together with intermediate results of EdgeUpdate and Aggregate to reduce communication costs in our proposed dynamic redundancy-eliminated message passing algorithm.

3) Algorithm Design

As shown in Algorithm 1, there are four phases in our proposed dynamic redundancy-eliminated message passing algorithm for GNNs: Dispatch, redundancy-eliminated EdgeUpdate, redundancy-eliminated Aggregation, and VertexUpdate.

Dynamic Dispatch. To obviate the workload irregularity and imbalance, we introduce a dynamic dispatching mechanism. The Dispatch phase aims at dispatching the EdgeUpdate work and Aggregation work by traversing all redundant set neighbors and original neighbors, and all of the original vertices in each redundant set neighbor based on their state information. (i) EdgeUpdate workload. If the EdgeUpdate intermediate results of traversed vertices are not stored, these vertices will be assigned and reserved to vertex list *euList* for the following EdgeUpdate. If the EdgeUpdate intermediate results of traversed vertices have been stored, intermediate results of traversed vertices are only required to be fetched for the following Aggregation. (ii) Aggregation workload. For each

redundant set neighbor, we need to conduct aggregation for all the EdgeUpdate results of each original vertex. For each target vertex, we need to conduct aggregation for all the redundant set neighbors and original neighbors. The Aggregation workloads are reserved in *aList*.

Redundancy-eliminated EdgeUpdate. The EdgeUpdate phase aims to conduct EdgeUpdate for each vertex in *euList* based on the edge weights. Owing to the fact that only the vertices that are not performed EdgeUpdate operations are dispatched into the *euList* in the dispatch phase, the EdgeUpdate could be eliminated. The results in this phase are combined with existing workload in *aList* to form new Aggregation work. In the meanwhile, the results of each edge that have been conducted EdgeUpdate are cached, so that they can be reused for executing EdgeUpdate for other target vertices.

Redundancy-eliminated Aggregate. The Aggregate phase aims to conduct Aggregate of each work in *aList*. Each work in *aList* is also a list which contains a set of vertices that are required to be aggregated. The results in this phase are combined with existing work in *aList* to form new Aggregation work. In the meanwhile, the intermediate result of each aggregation set are cached. During the execution of other target vertices which contains these aggregation sets, the results can be fetched and reused. Therefore, the aggregation redundancies of redundant aggregation sets are eliminated.

VertexUpdate. It combines the old vertex features of each target vertex and the aggregated features to produce new vertex features based on the vertex weights.

4) Redundancy-aware Bulk Execution

We utilize the redundancy-aware bulk execution to alleviate the Communication redundancy. The idea of the vertex interval and edge shard proposed in [5], [19], [31] is adopted to partition our redesigned graph structure. Then a specific partition is executed in a bulk. Therefore, vertex features within this bulk only need to be fetched once from the off-chip memory to the on-chip memory. This reduces the peak amount of on-chip memory needed for computation since only a portion of the graph has to be loaded once. Moreover, during the message passing process, the EdgeUpdate operation of a specific vertex, and the Aggregate operation of a specific redundant aggregation set are executed once and then be reused for all target vertices in this bulk.

IV. ARCHITECTURE DESIGN OF REGNN

ReGNN is an accelerator architecture for low-latency GNN execution that can eliminate EdgeUpdate and Aggregation redundancy. It is also a configurable pipelined architecture that can be adapted to support different variants of GNNs.

A. Architecture Overview

Fig. 3 illustrates the architecture of ReGNN, which is designed to support the proposed dynamic redundancy-eliminated message passing algorithm efficiently and reduce the computation irregularities. ReGNN contains three core

Algorithm 1 Redundancy-eliminated message passing for GNNs.

```

1: for  $k$  max number of layers  $K$  do
2:   for each  $v \in V$  do
3:      $\triangleleft$  Dispatch
4:      $(euList, aList) \leftarrow \text{Dispatch}(v)$ 
5:      $\triangleleft$  EdgeUpdate (Optional)
6:     for each reserved vertex  $u$  in the  $euList$  do
7:        $e_{(u,v)} \leftarrow \text{EdgeUpdate}(h_v^{k-1}, h_u^{k-1}, W_{edge})$ ;
8:       Store  $e_{(u,v)}$  and modify the state information of  $u$ .
9:       Shuffle  $e_{(u,v)}$  into  $aList$ ;
10:    end for
11:     $\triangleleft$  Aggregation
12:    for each vertex list  $vl$  in the aggregation work list of  $aList$  do
13:      for each reserved  $u$  in the neighbor list of  $vl$  do
14:         $a_v^k \leftarrow \text{Aggregate}(a_v^k, e_{(u,v)}, h_u^{k-1})$ ;
15:      end for
16:      Store  $e_{(u,v)}$  and modify the state information of  $u$ .
17:      Shuffle  $e_{(u,v)}$  into  $aList$ ;
18:    end for
19:     $\triangleleft$  VertexUpdate
20:    if  $v$  is reserved then
21:       $h_v^k \leftarrow \text{VertexUpdate}(a_v^k, h_v^{k-1}, W_{vertex})$ ;
22:    end if
23:  end for
24: end for

```

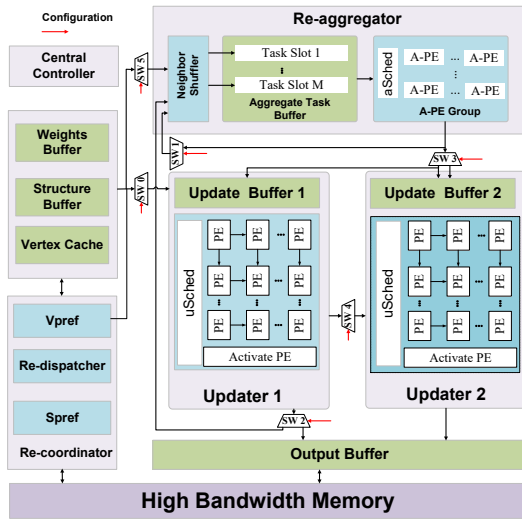


Fig. 3. Architecture overview of ReGNN.

execution engines: *Re-coordinator*, *Re-aggregator*, and *Flexible Updaters*. A *Central Controller* is designed to configure the pipelined execution flow of these engines to support various GNNs, that have EdgeUpdate, ER and AR or not. The execution pipeline of these engines accords with the proposed dynamic redundancy-eliminated message passing algorithm, which can eliminate redundancies in one phase.

Redundancy-eliminated Coordinator (Re-coordinator).

The *Re-coordinator* identifies the computation redundancy, decouples and assembles the irregular computations of redundancy-aware EdgeUpdate and Aggregation tasks into multiple regular EdgeUpdate and Aggregation tasks to exploit the intra-task regularity with highly parallelized computing engines. Specifically, only the vertices that are not conducted EdgeUpdate operations are sent to the *Updaters* for execution.

In terms of aggregation tasks, only those for target vertices and aggregation redundancy sets that are not aggregated will be sent to the *Re-aggregator* for execution.

Re-coordinator contains a Structure Prefetcher (Spref), a Re-dispatcher, and a Vertex Prefetcher (Vpref), where Spref aims at prefetching the structure and state information from High Bandwidth Memory (HBM), Re-dispatcher aims at dispatching irregular computation, and Vpref aims at prefetching the vertex properties to corresponding buffers for execution. Re-dispatcher dispatches the EdgeUpdate work to *Updaters* and Aggregation work to *Re-aggregator* for specific target vertices by traversing all the redundant set neighbors and original neighbors, and all the original vertices in each redundant set neighbor based on their state information.

Redundancy-eliminated Aggregator (Re-aggregator). *Re-aggregator* is designed to support our proposed redundancy-eliminated Aggregate algorithm, which can execute both aggregation tasks for target vertices and aggregation redundancy sets. It contains a *Neighbor Shuffler*, an *Aggregator Buffer* and an *Aggregate Engine*.

The design of *Neighbor Shuffler* enables *Re-aggregator* to reuse the original vertices features, intermediate results of EdgeUpdate and Aggregate for redundancy aggregation sets in a bulk to reduce the Communication redundancy on-the-fly. Based on the *Neighbor Shuffler*, irregular aggregation tasks are assembled to regular aggregation tasks stored in the *Aggregator Task Buffer*.

Aggregate Engine fetches the aggregation tasks in the *Aggregator Task Buffer* and executes accordingly. Moreover, it can be configured to store intermediate results of Aggregate for redundancy aggregation sets to the cache or the DRAM so that these intermediate results can be reused for the subsequent execution of other vertices.

Flexible Updaters.

ReGNN has 2 *Updaters* to flexibly support GNN variants. For GNN variants with EdgeUpdate stage, *Updater 1* performs the EdgeUpdate, while *Updater 2* performs VertexUpdate. *Updater 1* receives and executes the update tasks dispatched from the *Re-coordinator*. If it is for EdgeUpdate and the GNN variant under execution has ER, we can configure it to store the intermediate EdgeUpdate results. Moreover, the *Updaters* support multi-granularity usage. They can be combined together for VertexUpdate to support GNNs without EdgeUpdate and further avoid *Updaters* going idle. We adopt the well-known systolic array [14] for MVM operations in the *Updaters*.

Central Controller. ReGNN can flexibly configure the pipelined execution flow of these engines to support various GNNs. It consists of a *Central Controller* and six micro-configurable *Switches* (SW0-SW5). The *Central Controller* can issue control commands asynchronously to manage all configurable *Switches*. These micro-configurable switches can be configured in line with the control command to match the target execution flow.

Buffer Management. To reduce the latency of data access, we

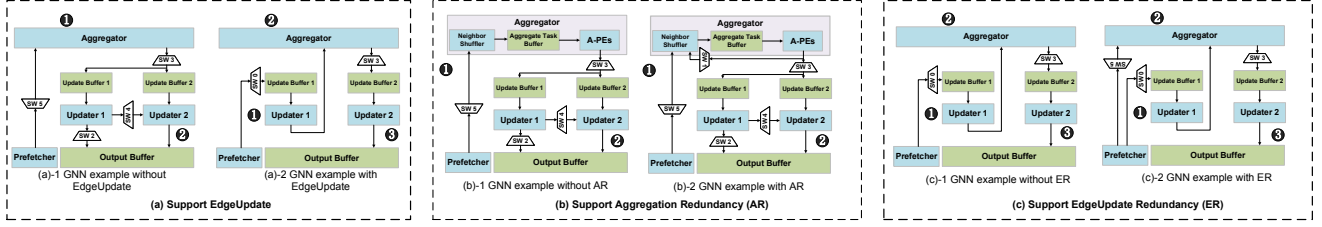


Fig. 4. Typical examples of flexible pipelined execution flow to support GNN variants. (a) Support GNNs with EdgeUpdate. (b) Support GNNs with AR. (c) Support GNNs with ER.

TABLE II

CONFIGURATION OF SWITCHES. "x" MEANS THE CONTROL SIGNAL WITHOUT OUTPUT. "x" DENOTES IRRELEVANT SIGNAL. THE VALUE 0 AND 1 MEAN GNN WITHOUT EDGEUPDATE AND WITH EDGEUPDATE RESPECTIVELY. THE VALUE 0 AND 1 MEAN GNNs WITHOUT AR AND WITH AR RESPECTIVELY. THE VALUE 0 AND 1 MEAN GNNs WITHOUT ER AND WITH ER RESPECTIVELY.

Switch	Input	Configura-tion Word			Output
		s1	s2	s3	
SW 0	Prefetcher	0	x	x	-
		1	x	x	Update Buffer 1
SW 1	Aggregator	x	0	x	-
		x	1	x	Neighbor Shuffler
SW 2	Updater 1	0	x	x	Output Buffer
		1	x	x	Neighbor Shuffler
SW 3	Aggregator	0	x	x	Update Buffer 1,
		1	x	x	Update Buffer 2
		0	x	x	Update Buffer 2
SW 4	Updater 1	0	x	x	Update Buffer 2
		1	x	x	-
SW 5	Prefetcher	0	x	x	Neighbor Shuffler
		1	x	0	-
		1	x	1	Neighbor Shuffler

employ on-chip buffers to cache various data and intermediate results to improve data reuse. In addition to *Updater Buffers* and *Aggregator Buffers*, there are *Weights Buffers*, *Structure Buffers* and *Intermediate Cache*. *Weights Buffer* is utilized to store weights of GNNs, *Structure Buffer* is utilized to store structure and state information, while *Intermediate Cache* is utilized to store the intermediate results generated by *Re-aggregator* and *Updaters*. To hide the access latency, we adopt double-buffering technology to allow overlapping execution of different operations for all the buffers.

B. Configurable Pipelined Execution Flow

As discussed before, the execution of GNN algorithms varies depending on the existence of EdgeUpdate, ER and AR. We use a 3-bit configuration word (s_1 , s_2 and s_3 shown in Table II), representing the existence of EdgeUpdate, ER and AR respectively, to denote a GNN variant. There are 6 micro-configurable *Switches* to configure the pipelined execution flow of the engines based on the configuration word to support various GNNs. Through this mechanism, the execution flow can be configured to accord with different pipelines of redundancy-eliminated message passing algorithms of different GNN variants. The configuration of these switches is shown in Table II.

Support EdgeUpdate. Fig. 4(a)-1 depicts the execution flow of GNN variants without EdgeUpdate which contains 2 pipelined phases: (i) Aggregate performs aggregate operations

with the inner *Neighbor Shuffler* and the *Aggregate Engine*. (ii) VertexUpdate, which uses the update module assembled by two *Updaters* to update the target vertices.

To support GNN variants with EdgeUpdate shown in Fig. 4(a)-2, which have an additional phase before aggregation: updating features for each edge, ReGNN needs be re-configured, which contains 3 phases: (i) EdgeUpdate, (ii) Aggregate, and (iii) VertexUpdate. At this time, the two *Updaters* are configured to perform EdgeUpdate and VertexUpdate respectively.

Support Aggregation Redundancy. If AR exists in GNN algorithms, the *Re-aggregator* aggregates both target vertices and redundant aggregation sets, and the intermediate aggregation results of sets are reused for aggregating target vertices. In doing so, the *Re-aggregator* can be re-configured to send the intermediate aggregation results of sets to the *Neighbor Shuffler* for on-chip reuse. Fig. 4(b)-2 depicts the re-configured execution flow when AR exists. Compared with those without AR in Fig. 4(b)-1, there is an additional path between the *Aggregate Engine* and the *Neighbor Shuffler*.

Support EdgeUpdate Redundancy. If ER exists in the EdgeUpdate stage, EdgeUpdate could have been performed or not for each source vertex. If the EdgeUpdate has been performed, the intermediate EdgeUpdate result of this source vertex should be transferred to the *Neighbor Shuffler* for the subsequent aggregation. Otherwise, the vertex feature of the source vertex should be sent to the *Updater 1* for EdgeUpdate. Fig. 4(c) shows that, for GNNs where there is no ER (Fig. 4(c)-1), the *Re-coordinator* only transfers the vertex feature to *Updater 1* for EdgeUpdate. If ER exists, as shown in Fig. 4(c)-2, the SW5 is re-configured to send the intermediate result to the *Re-aggregator*.

C. Design of Re-coordinator

The designed *Re-coordinator* is responsible for fetching the vertex features and structure information to establish the inter-task redundancy-aware execution pipeline for specific target vertices.

Handling Workload Irregularity. As we have discussed above, both the EdgeUpdate and the Aggregation stage of ReGNN, incur significant irregular computing and memory patterns. This irregularity leads to that the off-chip vertex features access can only be dynamically determined during the message passing. Nevertheless, the computing units such

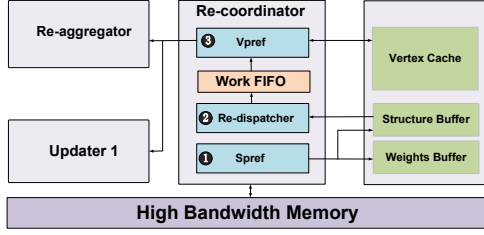


Fig. 5. Workflow of the Re-coordinator.

as the *Re-aggregator* and the *Updaters* are designed for highly paralleled and regular execution, therefore unable to cope with this irregularity in the message passing.

We design the *Re-coordinator* to bridge this gap and decouple the irregular redundancy-eliminated tasks in multiple EdgeUpdate and aggregation tasks. The *Re-coordinator* traverses through the given redundancy-aware graph structure and obtains the features with the exact vertex feature prefetch technique. For the EdgeUpdate tasks, the *Re-coordinator* only prefetches the vertex features that have not been performed EdgeUpdate, and assigns the task to the *Updaters*. For the aggregation tasks, the *Re-coordinator* prefetches the features according to whether the neighbor is a redundant aggregation set and whether it has been aggregated for AR elimination, and then assigns tasks that need to be aggregated to the *Re-aggregator*.

We also propose an exact prefetching technique during which the necessary data will be prefetched given the deterministic prefetching address and amount to prefetch. All the sequentially accessed data such as vertex features can be prefetched exactly.

Workflow of Re-coordinator. Fig. 5 illustrates the workflow of Re-coordinator. (i) *Spref*. This step is responsible for fetching the weights, structure information to the buffers. (ii) *Re-dispatcher*. Based on the structure information cached in the Structure Buffer, *Re-dispatcher* aims at dispatching EdgeUpdate tasks and Aggregation tasks by traversing all the redundant set neighbors and original neighbors, and all the original vertices in each redundant set neighbor based on their state information. The dispatching algorithm is detailed in Section III-B3. The workload is inserted into the *WORK FIFO*, which contains the exact source addresses of vertex properties, and the destination addresses, such as the addresses in the *Re-aggregator* or the *Updater 1*. (iii) *VPref* is responsible for fetching the vertex properties in High Bandwidth Memory.

D. Design of Redundancy-Eliminated Aggregator (*Re-aggregator*)

Re-aggregator is designed to support our proposed redundancy-eliminated Aggregate algorithm. Moreover, it can also reuse the original vertices features, intermediate results of EdgeUpdate and Aggregate in a bulk to alleviate the Communication redundancy on-the-fly.

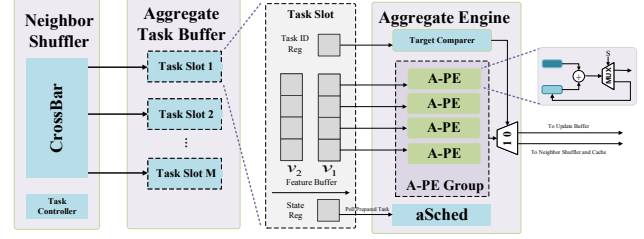


Fig. 6. Illustration of the architecture design of Re-aggregator.

1) Design Overview of Re-aggregator

Re-aggregator receives aggregation tasks assigned by the *Re-coordinator* and aggregates neighbor features for target vertices. The design of the *Re-aggregator* is shown in Fig. 6. It contains a *Neighbor Shuffler*, an *Aggregate Task Buffer* and an *Aggregate Engine*, which work in pipeline to achieve the aforementioned goals as follows.

The pipeline of *Re-aggregator* involves two distinct stages. (i) Each neighbor feature (including the intermediate activation of redundant set neighbors) within a bulk is loaded only once from DRAM by the *Re-coordinator*. Then all the neighbor features are shuffled and reused into multiple task slots in the *Aggregate Task Buffer* by the *Neighbor Shuffler*. Each task slot is assigned to an aggregation task which can aggregate a redundant aggregation set or a target vertex. (ii) In the aggregate stage, the *Aggregate Engine* first polls the task slot of redundant aggregation set and performs the aggregation. Then the aggregation results are sent to the *Neighbor Shuffler* and shuffled to slots of target vertices with this set neighbor for the following execution. This mechanism ensures that each redundant aggregation set is aggregated only once and then the intermediate results can be reused for target vertices in this bulk. In the meanwhile, the intermediate results are also sent to the cache and DRAM, which allows them to be reused for other target vertices. Once all the slots of redundant aggregation sets are aggregated, the *Aggregate Engine* will proceed to aggregate the remaining slots of specific target vertices.

Note that although the whole aggregations for specific target vertices are irregular, the features in each task slot are aligned, which facilitates utilization of the intra-task regularity by the highly paralleled *Aggregate Engine*. An illustrative example is shown in Fig. 7. Through our designed *Re-aggregator*, Communication redundancy within a bulk can be eliminated, and intermediate results of redundant aggregation sets can be reused.

2) Neighbor Shuffler

The *Neighbor Shuffler* eliminates the communication redundancy via reusing the intra-bulk features. It also shuffles and aligns the features into multiple task slots for highly paralleled execution of *Aggregate Engine*. Each aggregation task with a task ID is assigned to a *task slot*. Each task slot contains a *target ID reg* which stores the assigned task ID, a *feature buffer* to store neighbor features and a *state reg* to denote whether this task is prepared. An $N \times M$ crossbar

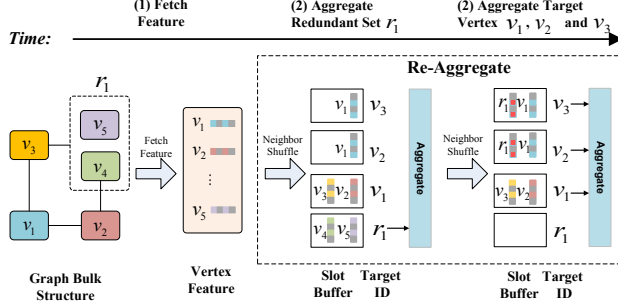


Fig. 7. Illustration of the workflow of Re-aggregator. (i) The vertex features of a graph bulk are loaded on-chip only once and the overlapping neighbor (v_1) are duplicated to the slots of v_2 and v_3 to avoid the communication redundancy. (ii) The un-aggregated set r_1 are aggregated first and its intermediate result is shuffled to the slots of target vertices v_2 and v_3 ; the intermediate result is also cached for further reuse. (iii) The target vertices v_1 , v_2 and v_3 are aggregated.

is responsible for shuffling the neighbor features that belong to an aggregation task to its assigned *task slot* based on the mapping between the task ID and the output ports. N input ports connect the *Re-coordinator*, the *Updater*, and the *Aggregate Engine*, which constitute the source of neighbor features, while M output ports connect M separate *task slots*. The input neighbor features of various tasks are shuffled and aligned in the assigned *task slots*. The features that belong to an overlapping neighbor will be duplicated and reused to the corresponding slots. After all the neighbor features of an aggregation task have been shuffled to its *task slot*, this *task slot* is set that it has been prepared and ready for the *Aggregate Engine* to aggregate.

3) Aggregate Engine

As all irregular aggregation has been decoupled into multiple aggregation tasks which are regularly aligned in the *task slots*. We further design the *Aggregate Engine* to execute these tasks with high parallelism. As shown in Fig. 6, the *Aggregate Engine* contains an aggregate scheduler (*aSched*), an *A-PE group* and a *target comparer*. The aggregate scheduler (*aSched*) guarantees the pipeline execution and assigns the workloads to *A-PE group* which works in a task disperse aggregation mode similar to HyGCN for workload balance and task-level parallelism. The *aSched* first polls the *state reg* of the *task slots*, selects a batch of prepared tasks and sends the features to the *A-PE group* for aggregation. The number of a batch depends on the dimension of features and the computation capacity of the *A-PE group*. After the *A-PE group* finishes their workloads, the aggregated results will be transferred according to the task type. As IDs of the target vertex and redundant aggregation set belong to distinct numerical intervals, we design a *task comparer* to evaluate the task type and determine where to send the results.

E. Design of Flexible Updaters

The update operations, whether it be EdgeUpdate or VertexUpdate, utilize Multilayer Perceptron (MLP) to update vertex

features. The classic systolic array [14] is adopted to construct two *Updaters*. As shown in Fig. 8, the *Updaters* can configure their execution mode with the *SW4* to support the two main categories of GNNs with no idle resource.

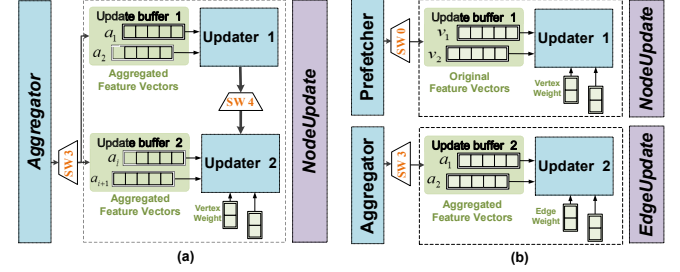


Fig. 8. Execution modes of *Updaters*. (a) Execution mode for GNNs with *EdgeUpdate*. (b) Execution mode for GNNs without *EdgeUpdate*.

Execution Mode for GNNs with EdgeUpdate. As illustrated in Fig. 8(a), the *Updaters* work independently to support different update operations. Each of them performs the matrix-vector multiplication (MVM) operation separately. In this case, *Updater 1* performs *EdgeUpdate*, where the original features vectors of a vertex set along with the weight parameter matrix of *EdgeUpdate* shifted into the MVM systolic array. *Updater 2* executes *VertexUpdate* which is also MVM operation between the aggregated feature vectors of a set of vertices with the weight parameter matrix of *VertexUpdate*. With this mode, the weight parameters are shared among the *Updaters* and only reused within modules. We can configure it to store the intermediate results of *Updater 1* so that these intermediate results could be reused for the subsequent execution of other target vertices.

Execution Mode for GNNs without EdgeUpdate. For the GNN variants without *EdgeUpdate*, the *Updater 1* will be idle when *Updater 1* stops running. Nevertheless, besides working separately, these systolic modules can be further assembled together to form a large systolic module which enables a larger set of elements to perform MVM simultaneously. Leveraging this characteristic for the GNN variant without *EdgeUpdate*, we utilize the *SW 4* to combine *Updater 1* and *Updater 2*, which form a large *Updater* to perform *VertexUpdate*, as shown in Fig. 8(b). In this manner, all *Updaters* can reuse the weight parameters. Moreover, unlike the independent working model mentioned above, a larger number of vertices can be shifted into the combined *Updaters* and processed in parallel.

With such execution modes, these *Updaters* are always busy, and not restricted by the GNN's execution flow. This effectively tackles the issue of idle computing resources and improves resource utilization.

F. Configurable Cache Management

To further reduce the off-chip memory communication, we design a cache management scheme to cache the vertex properties in the on-chip memory. Our cache management scheme is configurable for supporting different kinds of GNN variants. (i) For the variants of GNNs that have ER, we use

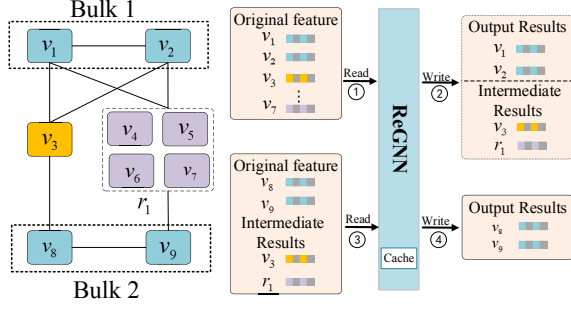


Fig. 9. An illustrative example with ReGNN.

the vertex cache to store intermediate results of EdgeUpdate. For the variants that do not have ER, we cache the original vertex properties. (ii) For the variants of GNNs that have AR, we also use the vertex cache to store intermediate aggregation results of redundant sets. To improve the cache hit rate, we take into consideration both the vertex degree information and the number of elements in the redundant set. Some cache entries are reserved for high-priority vertices. The priority of an original vertex is set as the degree of the vertex, while the priority of a redundant set is the number of its elements.

G. Analysis of Redundancies Elimination with ReGNN

In this section, we analyze how the computation and communication redundancies in message passing for GNNs are eliminated by utilizing our ReGNN. An end-to-end example with execution of two bulks is shown in Fig. 9. We conduct message passing for target vertices v_1 and v_2 for bulk 1, and for target vertices v_8 and v_9 for bulk 2. We also assume that the EdgeUpdate stage, the ER and AR all exist.

Analysis of Computation Elimination. ReGNN can eliminate the computation redundancies in message passing through reusing the intermediate EdgeUpdate results and aggregation results of redundant aggregation sets. (i) Without EdgeUpdate redundancy elimination, each of vertices v_3-v_7 will be redundantly performed EdgeUpdate 3 times as each has 3 edges. By contrast, ReGNN performs EdgeUpdate for each vertex only once when processing bulk 1 and later reuses the EdgeUpdate results among all the target vertices. (ii) Without aggregation redundancy elimination, as v_4-v_7 are shared by targets v_1, v_2 and v_9 , each of them will be aggregated 3 times. In ReGNN, v_4-v_7 is treated as a redundant set and aggregated only once when processing bulk 1 before the aggregation result of r_1 is reused in the processing of bulk 2.

Analysis of Communication Reduction. ReGNN can reduce the Communication redundancy in message passing. (i) With the bulk execution and neighbor shuffle techniques, the DRAM accesses to the original features of v_3-v_7 are not required to be read repeatedly for v_1 and v_2 respectively when processing bulk 1, while these DRAM accesses can be merged for both v_1 and v_2 . (ii) The DRAM read operations for the features of redundant elements can be reduced across bulks. In bulk 1, ReGNN needs to access the origin features for v_1-v_7 for message passing and write back the VertexUpdate result for

v_1 and v_2 , together with the intermediate EdgeUpdate result for v_3 and the aggregation result for r_1 . When processing bulk 2, ReGNN reads the intermediate aggregation result of r_1 instead of original feature of v_4-v_7 , and the intermediate EdgeUpdate result of v_3 . The process introduces write overheads of intermediate results, but they are trivial compared with the reduction of read operations and computation for processing the following bulks. (iii) Third, the configurable cache management technique further reduces DRAM access by cache access.

V. EVALUATION RESULTS

A. Experimental Setup

Methodology. The performance and energy of ReGNN are measured using the following tools.

Accelerator Simulator. To measure the performance of ReGNN, we implement a validated custom cycle-accurate simulator to model the behavior of the hardware. The simulator models the microarchitectural behavior of each module, and integrates with Ramulator [15] which supports high-bandwidth memory (HBM) to estimate HBM timings and produce a command trace.

CAD Tools. For power and area measurements, we implement ReGNN in Verilog, and utilize the Synopsys Design Compiler (DC) with the TSMC 12 nm standard VT library for the synthesis, and estimate the power consumption using Synopsys PrimeTime PX. The area and power of on-chip memory are calculated by CACTI [25].

Evaluated GNN Models. To evaluate ReGNN, we utilize three most common and representative GNN models: GCN [18], GraphSage-Pool [10] and G-GCN [4]. Following the analysis in Section II-B, GCN has AR but do not has ER, G-GCN suffers from ER but not AR, while GraphSage-Pool has both AR and ER. For all models, we use the two-layer model architecture wherein the hidden dimension is 512.

PE Implementation. Our proposed ReGNN adopts programmable ALUs, which implements different operations depending on the properties of the GNN model. A-PE can be element-wise sum, max and mean. Activate PEs support common activation operations such as rule, sigmoid, etc. PEs in the each *Updater* are constructed as a systolic array, and each of PE is basically a MAC unit with some glue logic.

Datasets Statistics and Redundancy Analysis. The datasets used for evaluation are Cora, PPI, COLLAB, BZR, IMDB, and Reddit, which are the most widely used datasets in GNN research. Table III provides detailed statistics. About 14.4% to 75.5% of the aggregation operations are redundant. Moreover, the number of edges is much larger than that of vertices. Such statistics prove that eliminating Aggregation and EdgeUpdate redundancies has great potential to improve the performance.

Compared Baselines. We compare ReGNN with two different baselines. The first category is the state-of-the-art GNN framework PyTorch Geometric implemented on general-purpose

TABLE III
DATASET STATISTICS AND REDUNDANCY ANALYSIS

Dataset	Lable	Feature Length	Vertices	Edges	Redundant Aggregation Sets	Reduced Aggregation
Cora (CA)	7	1,433	2,708	10,556	365	14.4%
BZR (BR)	2	3	6,519	137,734	3,187	46.6%
PPI (PI)	121	50	14,755	225,270	8,572	58.7%
Reddit (RD)	41	602	232,965	114,615,892	1,280,342	75.5%
IMDB (IB)	3	1	19,502	197,806	8,810	37.2%
COLLAB (CB)	3	1	372,474	12,288,900	173,935	50.4%

TABLE IV
SYSTEM CONFIGURATIONS OF COMPARED ACCELERATOR.

	HyGCN	AWB-GCN	GCNAX	HaGNN	ReGNN
Compute	1GHz @ 32 SIMD 16 cores and 32×128 arrays	1GHz @ 4K PEs	1GHz @ 1×16 MAC Array	1GHz @ 512 32-bit ALUs and 32×128 PEs	1GHz @ 512 A-PEs, 2×16 P-PEs with radix-64 float tree adder and 2 systolic modules (each with 32×128 arrays)
On-chip Memory	22MB+128KB	12M	4MB	24MB	2MB (Edge), 2 MB (Weight), 4 MB (Output), 2×4 MB (Update), and 4 MB (Aggregate)
Off-chip Memory	256GB/s HBM 1.0	256GB/s HBM 1.0	256GB/s HBM 1.0	256GB/s HBM 1.0	256GB/s HBM 1.0

processors, termed as PyG-CPU and PyG-GPU. CPU platform is equipped with two Intel Xeon(R) CPU E5-2680 v3 CPUs and 500GB DRAM, and GPU platform is equipped with NVIDIA Tesla V100. The second category is the state-of-the-art GNN accelerators, including HyGCN [31], AWB-GCN [7], GCNAX [20] and HaGNN [1]. HyGCN, AWB-GCN, and GCNAX are designed only for GCN, and they do not support GNN models with EdgeUpdate, while HaGNN is applicable to general GNNs. To enable a fair comparison, we enhance the configuration of the baseline accelerators. For GCNAX, we configure it with a larger on-chip buffer (4MB). For HaGNN, we configure the same amount of computational resources as ReGNN. More detailed configurations are listed in Table IV.

B. Overall Results

1) Speedup

In this experiment, we adopt the total number of execution cycles as the performance metric. Fig. 10 depicts the performance of ReGNN compared with other baselines. For different accelerator baselines, ReGNN, on average, is 9.8×, 10.3×, 1.8×, and 1.1× faster than HyGCN, HaGNN, AWB-GCN and GCNAX respectively. Such improvements are achieved because the proposed dynamic redundancy-eliminated message passing and the architecture design in ReGNN can greatly reduce the redundant computations and communications in various stages of GNNs. First, in the aggregation stage, ReGNN avoids redundant aggregations and DRAM accesses for elements in the redundant aggregation set by reusing its intermediate aggregation result for GNN models with AR (GCN and GS-Pool). Second, in the EdgeUpdate stage, ReGNN avoids performing redundant EdgeUpdate computations for the source vertices for GNN variants with ER (GGCN and GS-Pool). Third, the proposed bulk execution scheme and the configurable cache management further improves the data reuse and saves DRAM bandwidth. Finally, the *Re-coordinator* handles the irregular workloads in the redundancy-eliminated message passing and supports a fine-grained inter-engine pipeline.

For PyG-CPU and PyG-GPU, the abundant memory accesses, which are caused by irregular graph structure and the synchronization overhead, lead to performance degradation on GNNs. They implement GNNs with the entire graph Laplacian matrix. Although the redundant EdgeUpdate operation can be avoided, they fail to leverage the intermediate aggregation results. Moreover, they run out of memory on large graphs such as RD. GCNAX and AWB-GCN improves the performance for GCN by balancing the workloads between PEs, but they do not reduce the redundant aggregations or support GS-Pool and G-GCN with EdgeUpdate. ReGNN significantly outperforms them on graphs with high aggregation redundancy except for CA in which only 14% of the aggregations are redundant. HaGNN supports GS-Pool and G-GCN but fails to take into account the ER and the AR, which limits its performance.

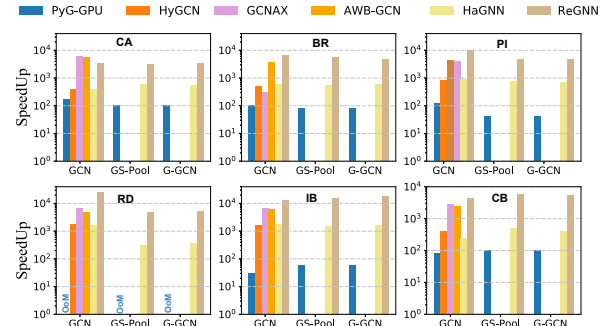


Fig. 10. Comparison of speedup over PyG-CPU.

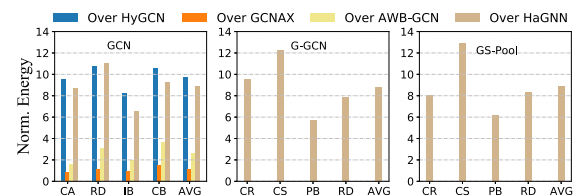


Fig. 11. Energy savings of ReGNN over the baselines.

2) DRAM Access

On average, ReGNN achieves $9.1\times$, $9.5\times$, $1.7\times$ and $1.2\times$ reduction on DRAM access than HyGCN, HaGNN, AWB-GCN and GCNAX respectively. The improvements over baselines are twofold: (i) For GNN baselines with AR, ReGNN can reuse the intermediate aggregation result of redundant aggregation sets and replace the DRAM accesses to multiple vertex feature of elements to one single aggregated feature of the set. The number of reduced memory accesses increases with the number of reduced aggregation operations. (ii) The proposed bulk execution and the configurable cache management techniques efficiently reuse the features of high degree vertices, thus eliminating significant DRAM communication.

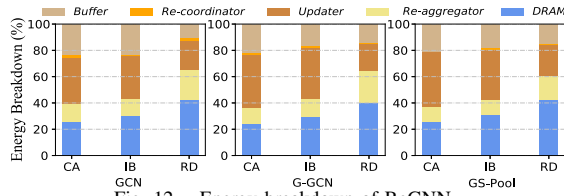


Fig. 12. Energy breakdown of ReGNN.

3) Energy Consumption

Fig. 11 provides the normalized energy efficiency comparison of ReGNN with other baseline accelerators. Overall, ReGNN achieves $9.8\times$, $2.6\times$, $1.1\times$, and $8.9\times$ energy savings compared with HyGCN, AWB-GCN, GCNAX, and HaGNN, respectively. First, ReGNN saves energy of the *Updaters* and *Re-aggregator* by reducing EdgeUpdate workloads and redundant aggregations performed on them respectively. Second, ReGNN saves energy of the DRAM as the proposed bulk execution and the cache management techniques greatly improve DRAM bandwidth utilization by exploiting the graph locality, and the redundancy-eliminated aggregation reduces the number of off-chip memory accesses to the vertex features in the aggregation stage. Fig. 12 depicts the energy breakdown of ReGNN on different benchmarks. The DRAM, *Re-aggregator*, *Updater*, *Re-coordinator* and the on-chip buffer consumes 32.8%, 16.5%, 30.2%, 2.8% and 17.7% of the total energy respectively on average. The control overhead in terms of energy of the *Re-coordinator* is very small to handle the irregular workloads and support a fine-grained inter-engine pipeline.

TABLE V
AREA BREAKDOWN OF REGNN.

Module	Area (%)
Updater	44.86
Re-aggregator	15.41
TotalBuffer	31.45
Re-coordinator	6.00
Others	2.28

4) Area Overhead Breakdown

The total area of ReGNN is only 11.5 mm^2 . Table V summarizes the area breakdown of the main components in ReGNN. We can observe that most of the area of ReGNN

is contributed by computational components such as *Re-aggregator* and *Updater*, which consume more than 60% of the total area, while Re-coordinator consumes only 6%. It indicates that the Re-coordinator is designed to introduce only a small amount of area overhead, but it can eliminate the redundancy in GNNs and significantly reduce the computational and memory access overhead of other engines. Comparing with HyGCN (7.8 mm^2), ReGNN is larger, mainly because we include one necessary *Updater* to efficiently support GNN variants with EdgeUpdate stage. In comparison with GCNAX (6.51 mm^2), ReGNN contains larger on-chip buffers. These additional hardware overheads are used to cache more intermediate results of redundant aggregation sets, which greatly improve execution efficiency.

C. Effect of Redundancy Elimination

We compare the performance of variants of ReGNN without our proposed redundancy elimination schemes with the original ReGNN to study the effect of all proposed redundancy elimination techniques. For GCN, only AR exists and we evaluate the performance of a variant of ReGNN without AR elimination scheme (ReGNN_NA) on it. For GGCN, only the ER exists so we study the performance of a variant of ReGNN without ER elimination scheme (ReGNN_NE) on it. For GS-Pool, both AR and ER exist, and we study the performance of the variants of ReGNN without AR elimination scheme (ReGNN_NA), without ER elimination scheme (ReGNN_NE), and without both schemes (ReGNN_NAE) on it. The results on various datasets are shown in Fig. 13.

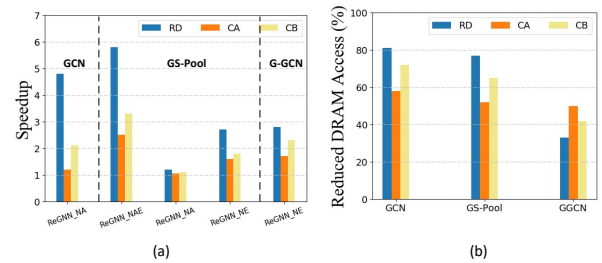


Fig. 13. Effect of redundancy elimination. (a) Speedup of ReGNN over variants without redundancy elimination schemes (denoted as ReGNN_NA, ReGNN_NAE, and ReGNN_NAE). (b) Reduced DRAM access with ReGNN.

Effect of Eliminating Aggregation Redundancy. As shown in Fig. 13, by just eliminating AR, ReGNN achieves $1.1\times$ to $4.8\times$ speedup over ReGNN_NA on GCN and GS-Pool. The performance increases as the relative number of redundant sets in the datasets increases. In RD where the reduced aggregation is the highest (over 75%), ReGNN also achieves the highest speedup. The average result is better on GCN ($2.7\times$) than GS-Pool ($1.1\times$), mainly because GCN has only 2 stages, while G-GCN has 3 stages. The relative execution time of the aggregation stage is much longer on GCN than on G-GCN. Therefore, optimizing the aggregation stage contributes to a higher speedup in GCN. This can be achieved by our proposed redundancy-eliminated message passing algorithm,

which reduces redundant aggregation by ensuring that all the redundant aggregation sets are only aggregated once and the intermediate results can be reused. Moreover, ReGNN reduces the off-chip memory access by replacing the memory access of multiple vertex features of all the elements in the redundant aggregation set with a single feature of intermediate aggregation results.

Effect of Eliminating EdgeUpdate Redundancy. Simply eliminating ER achieves $1.6\times$ to $2.8\times$ speedup over ReGNN_NE on G-GCN and GS-Pool. The speedup of G-GCN is better than GS-Pool, mainly because the G-GCN involves more MVM operations in its EdgeUpdate stage according to Table I. Additionally, the speedup is affected by the edge-vertex ratio of datasets. For graphs which have densely connected vertices and high edge-vertex ratios such as RD, eliminating ER leads to much better performance. One main reason is that the densely connected graphs tend to have more overlapped neighbors, which aggravates the EdgeUpdate redundancies. By eliminating both ER and AR, we achieve the best results over ReGNN_NAE on GS-Pool. The results have demonstrated ReGNN can greatly reduce the redundant MVM operations in the EdgeUpdate stage.

Effect of Reducing Communication Redundancy. In ReGNN, our designed mechanisms of reducing communication redundancy significantly reduce the DRAM accesses for all the three GNN models. As shown in Fig. 13(b), ReGNN achieves 58% of DRAM access reduction for the benchmarks on average. We bring the improvements from three aspects to improve the communication efficiency. First, the bulk execution technique helps reuse the original vertices features, intermediate results of EdgeUpdate and Aggregate in a bulk on-the-fly. The bulk size is chosen based on the capacity of aggregation buffer. A larger bulk size results in less execution time by reusing features more frequently during neighbor shuffle, but at the cost of larger area and energy overhead. Second, the configurable cache management reduces the off-chip memory communication by precisely caching the intermediate results. Third, ReGNN further reduces the DRAM accesses in the aggregation stage for GNN variants with AR (GCN and GS-Pool). Note that the reduced DRAM accesses of GS-Pool are fewer than GCN because GS-Pool needs to write the intermediate EdgeUpdate results back to DRAM.

D. Effect of Dynamic Redundancy-eliminated Message Passing

To verify the effect of our proposed dynamic redundancy-eliminated message passing in ReGNN, we design another simple architecture to support redundancy elimination, termed as ReGNN_NDP. We compare the performance of ReGNN_NDP with ReGNN. ReGNN_NDP uses separate phases to store and reuse the intermediate results of EdgeUpdate and Aggregate, which are discussed as follows. (a) The *Updater* performs the EdgeUpdate operation for all the vertices and writes back the intermediate EdgeUpdate results

to DRAM. (b) The *Aggregate Engine* performs the aggregation operations for the elements of all redundant aggregation sets and writes back the results. (c) The aggregation for all target vertices are performed by the *Aggregate Engine*. (d) The *Updater* calculates the VertexUpdate for all targets and generates the final output.

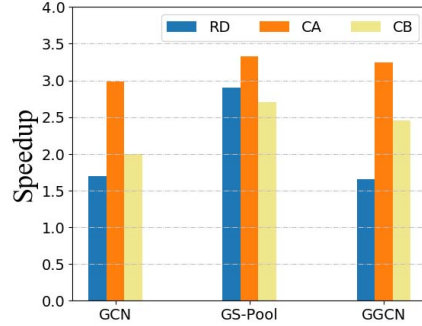


Fig. 14. Effect of dynamic redundancy-eliminated message passing on execution time. The speedup of ReGNN over ReGNN_NDP (variant of ReGNN without dynamic pipeline) is evaluated.

The experiment results are presented in Fig. 14. ReGNN is $1.7\times$ to $3.4\times$ faster than ReGNN_NDP. The speedup is achieved for the following reasons: (i) Because of the complex structure of graphs, the phases a, b, and c can hardly be pipelined, which causes severe idleness in ReGNN_NDP. For example, the *Aggregate Engine* is idle until the *Updater* finishes the EdgeUpdate on all the vertices. In contrast, ReGNN dynamically identifies the redundancy and handles the workload irregularity on-the-fly with the *Re-coordinator* and the *Neighbor Shuffler*. The vertex features are loaded from DRAM and assigned to *Updater* or *Re-aggregator* depending on the status bits of vertices. In this way, the inter-engine execution pipeline is established to avoid idleness. The *Updater* can work on a set of vertices which have not been performed EdgeUpdate, while the *Re-aggregator* work on another set of vertices which have been performed EdgeUpdate in parallel. (ii) ReGNN saves considerable DRAM accesses compared with ReGNN_NDP. For each phase of ReGNN_NDP, the intermediate results are written back to DRAM and further re-loaded in the next phase, while ReGNN can reuse the intermediate results of EdgeUpdate and Aggregate through neighbor shuffling and the configurable cache across different phases, therefore avoids a lot of redundant communications.

E. Offline Search Overhead Analysis

From the statistics in Table III, it is easy to observe that redundancies are prevalent in common graph datasets. With the special message passing pattern in GNN model, these redundant sets cause extensive computation and communication redundancies, which occupies a large amount of execution time. The proposed ReGNN can properly eliminate these redundancies despite some additional offline processing overhead. From Fig. 15, we can observe that the offline processing time is short compared with the training time.

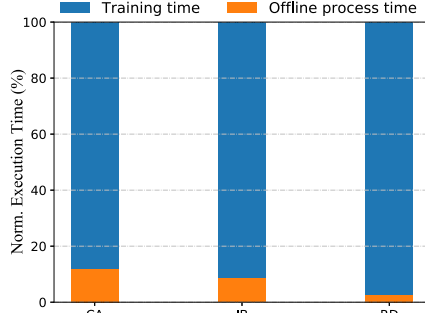


Fig. 15. Offline search time for redundant sets.

Therefore, although ReGNN introduces additional costs, it is trivial compared with the tradeoff of the resulting gain.

VI. RELATED WORK

To accelerate graph analytics, many customized accelerators were proposed to efficiently support these workloads [2], [9], [11], [23], [24], [26], [32]–[34]. We mainly discuss the work of accelerating graph analytics and GNNs which are most relevant to our work.

To accelerate the computations of GNNs, several customized accelerators [1], [16], [31] were proposed. They have given a comprehensive analysis of the necessities of designing new customized accelerators, and achieve much higher performance and lower energy consumption than GPUs and existing DNN accelerators for GNNs. A well-known pioneer work is HyGCN [31] which designs a hybrid hardware architecture with two processing engines to efficiently perform computation of GCNs [18]. AWB-GCN [7] is an architecture for accelerating GCNs and addresses the issue of workload imbalance in processing real-world graphs. To further improve the performance of GCN acceleration, Li et al. [20] proposed GCNAX whose dataflow is optimized for throughput and energy efficiency. These accelerators are designed only for GCNs, and thus they cannot eliminate EdgeUpdate redundancies. Moreover, these accelerators cannot identify and store the intermediate results of Aggregate, which makes it unable to eliminate Aggregate redundancies.

HaGNN [1] is the first accelerator architecture targeting GNNs. The authors analyzed popular GNN algorithms to uncover the fundamental building blocks for GNNs and then designs a novel architecture for GNNs which includes dedicated hardware units to efficiently execute the irregular data movement required for graph computation in GNNs, while also providing high compute throughput required by GNN models. HaGNN cannot identify and store the intermediate results of EdgeUpdate and Aggregate, which makes it unable to eliminate Aggregate redundancies.

VII. CONCLUSION

Existing typical GNNs utilize neighborhood messages passing mechanism based on neural networks that updates target vertex representations by aggregating feature messages from

neighboring source vertices. We identify that a naive implementation of the neighborhood aggregation leads to redundant computations and communications. In this work, we propose a novel redundancy-eliminated GNN accelerator, shortly termed as ReGNN. ReGNN is supported by algorithm and architecture co-design. We first propose an optimized redundancy-eliminated neighborhood message passing algorithm for GNNs that can identify and eliminate the computation redundancies. Then, a novel architecture is designed to support the proposed algorithm and transform the redundancy elimination into performance improvement. ReGNN performs the same computations and offers the same accuracy as traditional GNNs. To the best of our knowledge, ReGNN is the first accelerator that can eliminate computation redundancy in GNNs.

ACKNOWLEDGMENT

This work was partially funded by the National Key Research and Development Program of China under Grant 2020YFB2104000, by the National Natural Science Foundation of China under Grant 61902120, in part by the National Outstanding Youth Science Program of National Natural Science Foundation of China under Grant 61625202, by the Cultivation of Shenzhen Excellent Technological and Innovative Talents (Ph.D. Basic Research Started) RCBS20200714114943014, and by the Basic research of Shenzhen Science and technology Plan under Grant JCYJ20210324123802006.

REFERENCES

- [1] A. Auten, M. Tomei, and R. Kumar, "Hardware acceleration of graph neural networks," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2020, pp. 1–6.
- [2] V. Balaji and B. Lucia, "Combining data duplication and graph reordering to accelerate parallel graph processing," in *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing*, 2019, pp. 133–144.
- [3] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malininowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner et al., "Relational inductive biases, deep learning, and graph networks," *arXiv preprint arXiv:1806.01261*, 2018.
- [4] X. Bresson and T. Laurent, "Residual gated graph convnets," *arXiv preprint arXiv:1711.07553*, 2017.
- [5] Y. Chi, G. Dai, Y. Wang, G. Sun, G. Li, and H. Yang, "Nxgraph: An efficient graph processing system on a single machine," in *2016 IEEE 32nd International Conference on Data Engineering (ICDE)*. IEEE, 2016, pp. 409–420.
- [6] M. Fey and J. E. Lenssen, "Fast graph representation learning with pytorch geometric," *arXiv preprint arXiv:1903.02428*, 2019.
- [7] T. Geng, A. Li, R. Shi, C. Wu, T. Wang, Y. Li, P. Haghi, A. Tumeo, S. Che, S. Reinhardt et al., "Awb-gcn: A graph convolutional network accelerator with runtime workload rebalancing," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 922–936.
- [8] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," *arXiv preprint arXiv:1704.01212*, 2017.
- [9] T. J. Ham, L. Wu, N. Sundaram, N. Satish, and M. Martonosi, "Graphicionado: A high-performance and energy-efficient accelerator for graph analytics," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–13.
- [10] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in neural information processing systems*, 2017, pp. 1024–1034.

- [11] S. Hong, S. K. Kim, T. Oguntebi, and K. Olukotun, "Accelerating cuda graph algorithms at maximum warp," *Acm Sigplan Notices*, vol. 46, no. 8, pp. 267–276, 2011.
- [12] X. Huang, H. Cheng, L. Qin, W. Tian, and J. X. Yu, "Querying k-truss community in large and dynamic graphs," in *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*, 2014, pp. 1311–1322.
- [13] Z. Jia, S. Lin, R. Ying, J. You, J. Leskovec, and A. Aiken, "Redundancy-free computation for graph neural networks," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 997–1005.
- [14] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, 2017, pp. 1–12.
- [15] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible dram simulator," *IEEE Computer architecture letters*, vol. 15, no. 1, pp. 45–49, 2015.
- [16] K. Kinningham, C. Re, and P. Levis, "Grip: A graph neural network accelerator architecture," *arXiv preprint arXiv:2007.13828*, 2020.
- [17] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.
- [18] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *International Conference on Learning Representations*, 2017.
- [19] A. Kyrola, G. Blleloch, and C. Guestrin, "Graphchi: Large-scale graph computation on just a {PC}," in *10th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 12)*, 2012, pp. 31–46.
- [20] J. Li, A. Louri, A. Karanth, and R. Bunescu, "Gcnax: A flexible and energy-efficient accelerator for graph convolutional neural networks," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 775–788.
- [21] S. Liang, Y. Wang, C. Liu, L. He, L. Huawei, D. Xu, and X. Li, "Engn: A high-throughput and energy-efficient accelerator for large graph neural networks," *IEEE Transactions on Computers*, 2020.
- [22] L. Ma, Z. Yang, Y. Miao, J. Xue, M. Wu, L. Zhou, and Y. Dai, "Neugraph: parallel deep neural network computation on large graphs," in *2019 {USENIX} Annual Technical Conference ({USENIX}{ATC} 19)*, 2019, pp. 443–458.
- [23] T. Oguntebi and K. Olukotun, "Graphops: A dataflow library for graph analytics acceleration," in *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2016, pp. 111–117.
- [24] M. M. Ozdal, S. Yesil, T. Kim, A. Ayupov, J. Greth, S. Burns, and O. Ozturk, "Energy efficient architecture for graph analytics accelerators," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 166–177, 2016.
- [25] T. Shyamkumar *et al.*, "Cacti 5.0," 2007.
- [26] L. Song, Y. Zhuo, X. Qian, H. Li, and Y. Chen, "Graphr: Accelerating graph processing using rram," in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2018, pp. 531–543.
- [27] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," *International Conference on Learning Representations*, 2018.
- [28] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *International conference on machine learning*. PMLR, 2019, pp. 6861–6871.
- [29] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, 2020.
- [30] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" *arXiv preprint arXiv:1810.00826*, 2018.
- [31] M. Yan, L. Deng, X. Hu, L. Liang, Y. Feng, X. Ye, Z. Zhang, D. Fan, and Y. Xie, "Hygcn: A gcn accelerator with hybrid architecture," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 15–29.
- [32] M. Yan, X. Hu, S. Li, A. Basak, H. Li, X. Ma, I. Akgun, Y. Feng, P. Gu, L. Deng *et al.*, "Alleviating irregularity in graph analytics acceleration: A hardware/software co-design approach," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 615–628.
- [33] Y. Yang, Z. Li, Y. Deng, Z. Liu, S. Yin, S. Wei, and L. Liu, "Graphabcd: Scaling out graph analytics with asynchronous block coordinate descent," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 419–432.
- [34] M. Zhang, Y. Zhuo, C. Wang, M. Gao, Y. Wu, K. Chen, C. Kozyrakis, and X. Qian, "Graphp: Reducing communication for pim-based graph processing with efficient data partition," in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2018, pp. 544–557.
- [35] M. Zhang and Y. Chen, "Link prediction based on graph neural networks," in *Advances in Neural Information Processing Systems*, 2018, pp. 5165–5175.
- [36] M. Zhang, Z. Cui, M. Neumann, and Y. Chen, "An end-to-end deep learning architecture for graph classification," in *Thirty-Second AAAI Conference on Artificial Intelligence*, 2018.
- [37] Z. Zhang, P. Cui, and W. Zhu, "Deep learning on graphs: A survey," *IEEE Transactions on Knowledge and Data Engineering*, 2020.